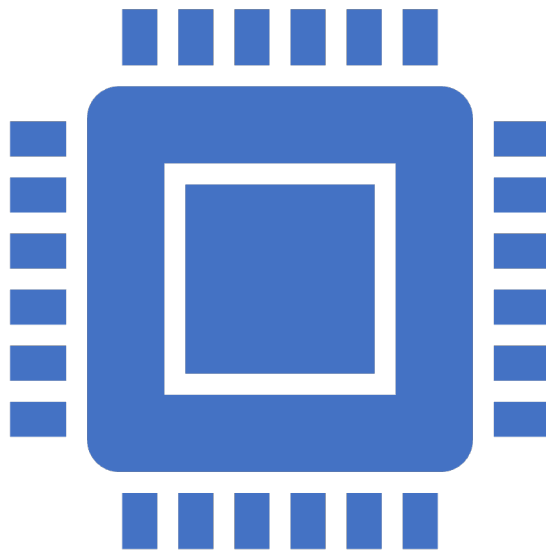




Оптимизация кода при помощи Numba (CPU и GPU)

Python никуда не торопится



1. Интерпретируемый, а не компилируемый язык
2. Динамическая типизация (очень удобно и универсально, но это сложно оптимизировать)
3. GIL (Global Interpreter Lock)

Процессы и потоки

Это две абстракции, которые возникли в результате долгого процесса эволюции.

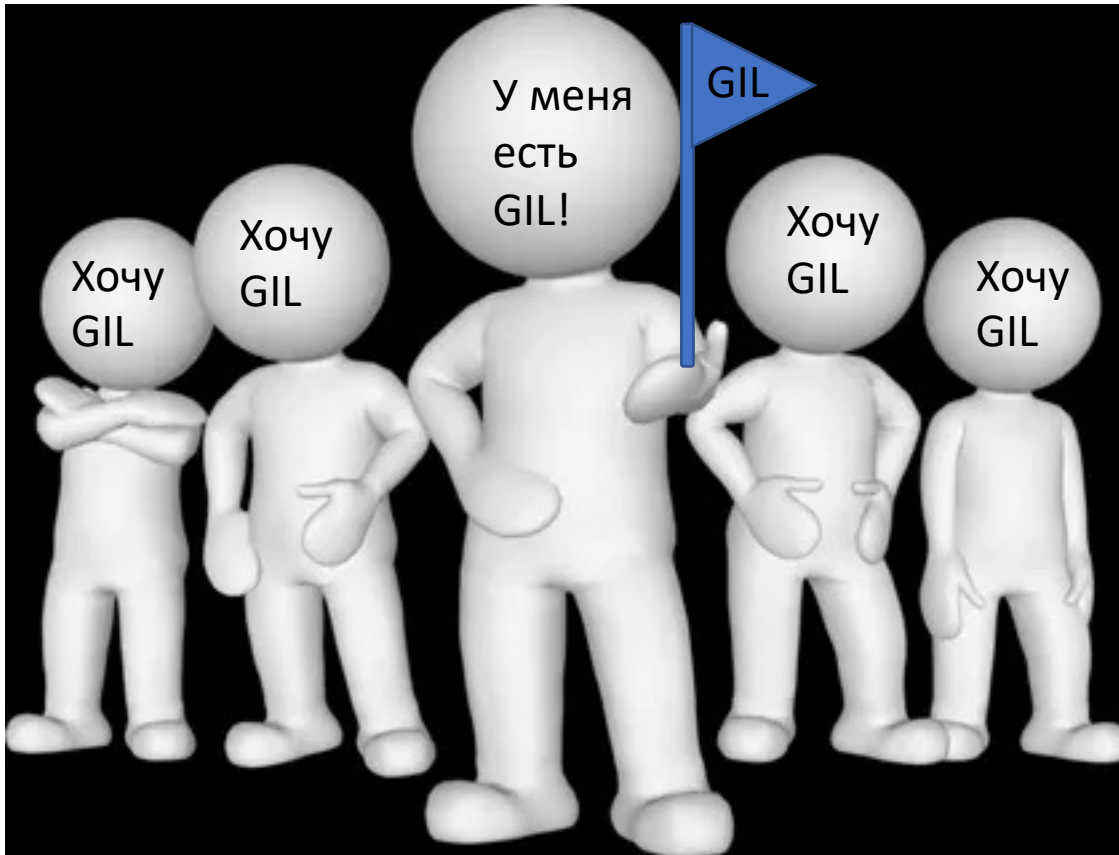
Процесс	Поток
Виртуальная изолированная память (думает, что она ему доступна вся)	Общая со всеми память
Свои ресурсы	Общие ресурсы
Никому не мешает	Мешает, думает, что он тут один

Немного истории

- Потоки в рамках одного процессора не могут работать параллельно;
- Процессор идет в память, смотрит байтик и что-то с ним делает;
- В рамках одного процессора работа потоков – пошаговая стратегия;
- Раз в 20-25 мс операционная система усыпляет один поток и пробуждает другой случайным образом;
- Потоки могут друг другу испортить участки памяти;
- В Java, C# есть «флажки»;
- В Python есть GIL.

Хочу GIL

Раньше были тики, но сейчас GIL активизируется раз в 5 мс.

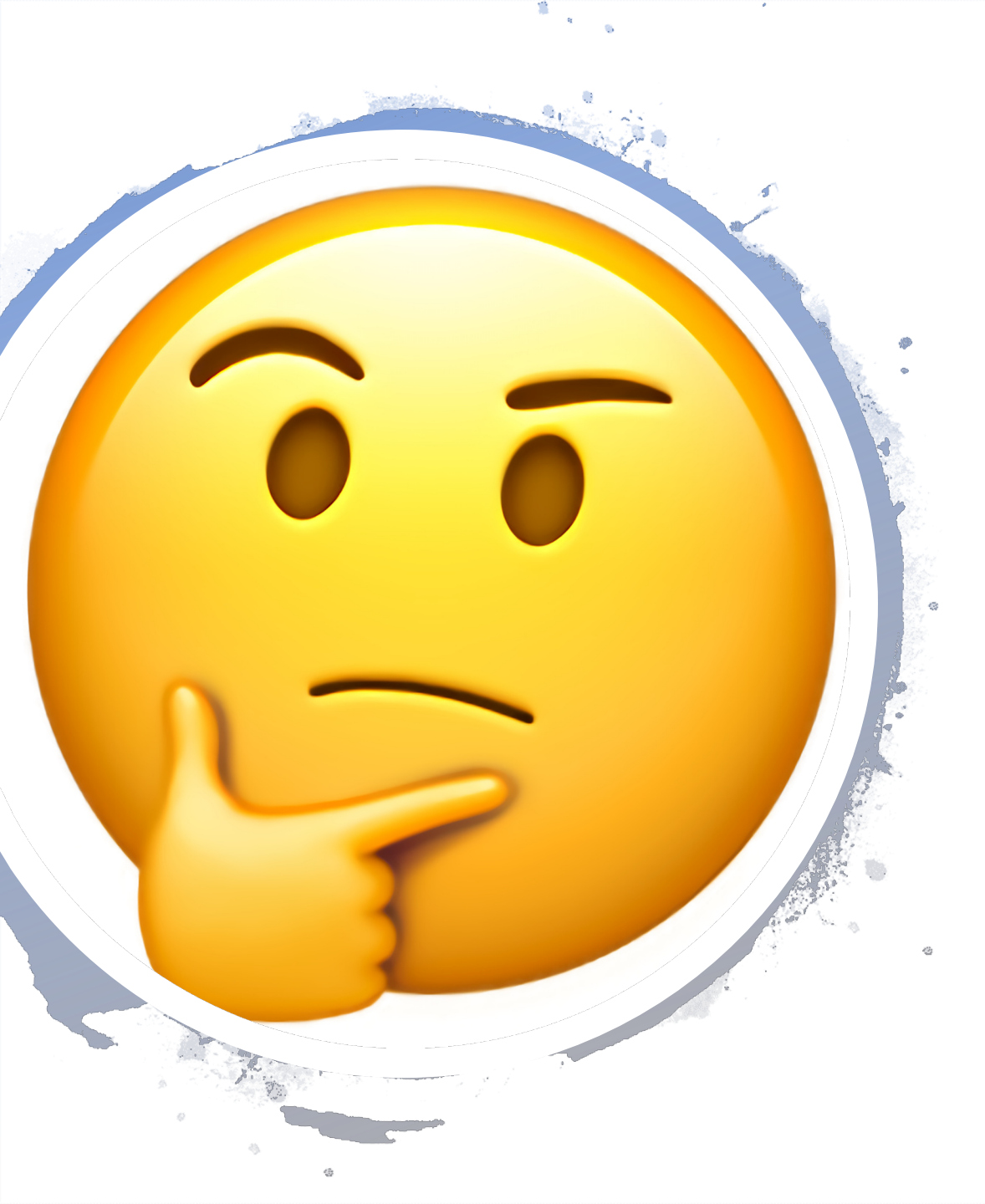


Хоть раз в 20-25 мс уснет тот поток, у кого поднят GIL, другой не будет работать. Через 5 мс активизируется GIL и разбудит того, у кого есть «флажок».

А еще есть исследования, что это может быть лучше, чем резервирование памяти.

Когда что использовать

CPU-bound – процессы;
I/O-bound – потоки.



СPython, что не так?

- СPython выполняет байт-код, а как в других языках?
- Java компилирует свою программу в «промежуточный» язык, виртуальная машина Java читает байт код и выполняет его JIT-компиляцию в машинный код. Такой же подход и у С#.
- «Промежуточные языки», виртуальные машины, машинный код...почему это быстрее?

В чем магия JIT-компиляции?

JIT-компилятор требует наличия промежуточного языка, чтобы была возможность разбивать код на фрагменты. Сам компилятор ничего не ускоряет, но позволяет проанализировать код и оптимизировать его участки.

JIT – Just-in-time компиляция в момент исполнения в отличии от AOT(Ahead-of-time).

Если какие-то операции часто выполняются, их надо оптимизировать!

JIT-компилятор кстати есть в интерпретаторе PyPy.

Недостатки:

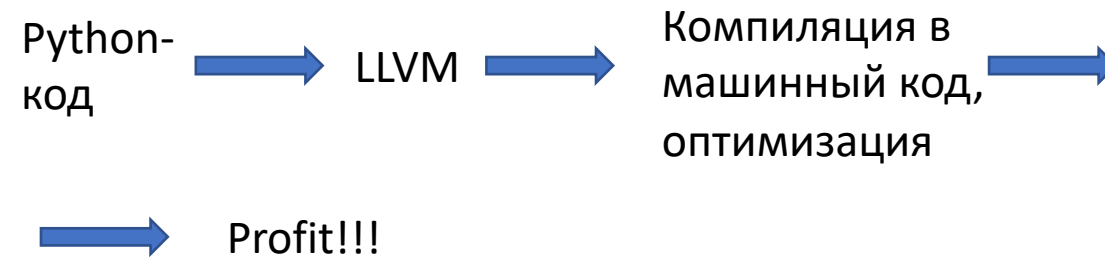
- Долго запускается
- Нужен «промежуточный язык»
- При строгой типизации работает быстрее за счет больших «предположений о программе» (спорный недостаток =))

Numba

Библиотека для оптимизации cpy bound задач. В отличие от PyPy не ставят задачей поддержку всех типов в Python.

Ускоряет не всю программу, а только отдельные ее части.

Использует LLVM(Low Level Virtual Machine), написанный на C++.



Декораторы Numba

@jit

Самый базовый декоратор, который использует Numba JIT компилятор для оптимизации. Numba сама анализирует код и решает, какие его части можно оптимизировать. Внутри генерируется код, обрабатывающий все значения как объекты Python и использующий Python C API для выполнения всех операций с этими объектами.

```
@jit
def compute_logit_numba(y):
    logit = 1 / (1 + math.exp(y))
    return logit
```


Декораторы Numba

@njit или @jit(nopython=True)

Режим компиляции Numba, который генерирует код, не имеющий доступа к Python C API. Этот режим компиляции создает код с наивысшей производительностью, но может взаимодействовать только с определенными типами объектов. Особенно любит NumPy и хорошо его оптимизирует, распараллеливает.

<http://numba.pydata.org/numba-doc/latest/reference/pysupported.html#pysupported-builtin-types>

<http://numba.pydata.org/numba-doc/latest/reference/numbysupported.html>

Параметры:

nogil - отключает GIL для nopython mode и позволяет использовать многопоточность.

parallel - позволяет автоматически распараллеливать код там, где это возможно

(<https://numba.readthedocs.io/en/stable/user/parallel.html#numba-parallel>)

```
@njit
def compute_logit_numba_2(y):
    logit = 1 / (1 + math.exp(y))
    return logit
```

ты функции

Декораторы Numba

@vectorize

Данный декоратор позволяет создавать универсальные функции ufuncs оперируя синтаксисом numpy, достигая производительности, как на ufuncs написанных на языке C. Функция работает с данными не как с массивами, а отдельно с каждым скаляром, создавая оптимальные циклы для итерирования. Данный вариант не всегда будет быстрее, чем njit, но он позволяет работать с GPU. По умолчанию параметр target='cpu', но можно выставить 'parallel' и 'cuda'.

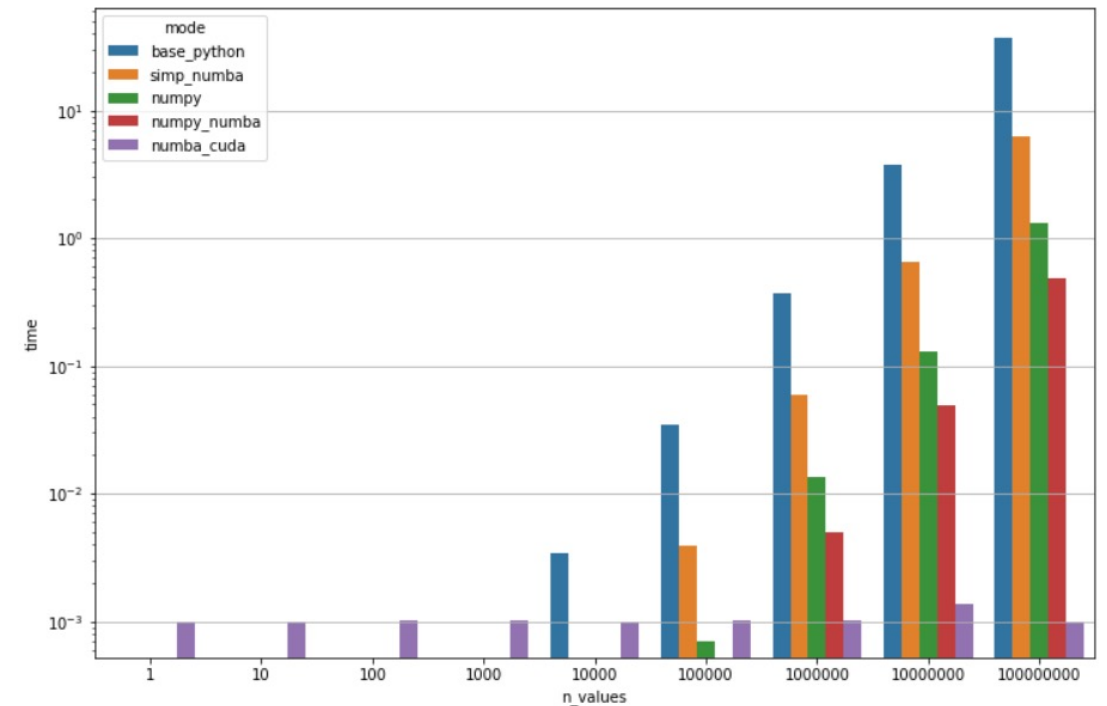
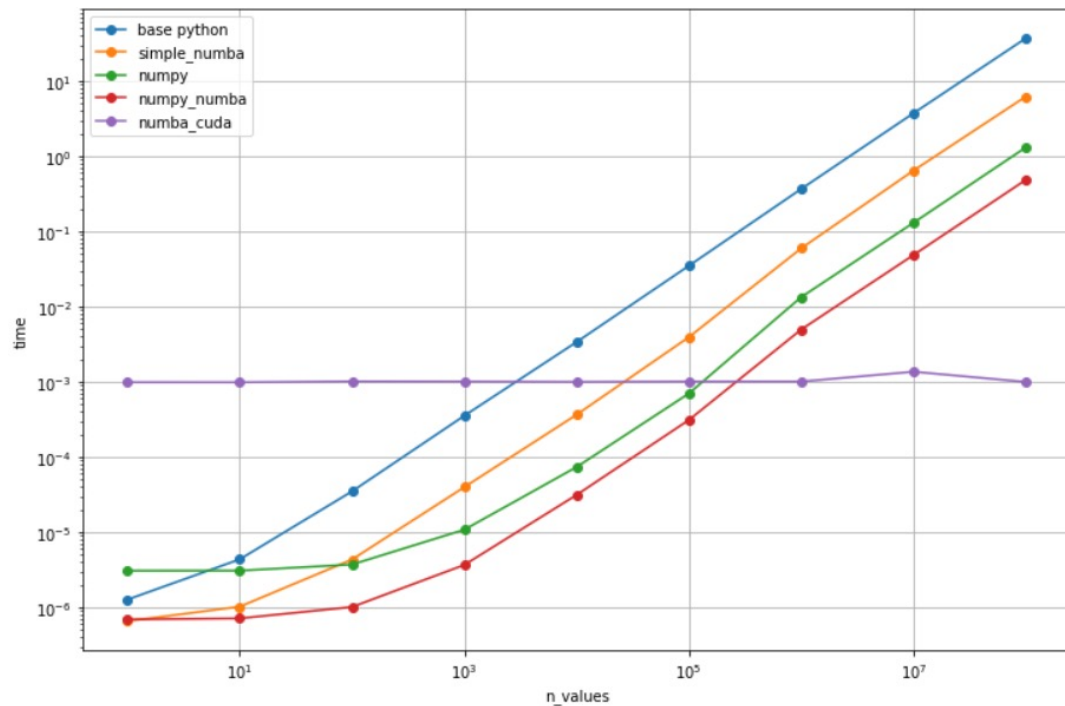
Не все типы данных и библиотеки поддерживаются одновременно на CPU и GPU.

<http://numba.pydata.org/numba-doc/latest/cuda/cudapysupported.html>

```
@vectorize([float64(float64)], nopython=True)
def logistic_numba_v(y):
    return 1 / (1 + np.exp(y))

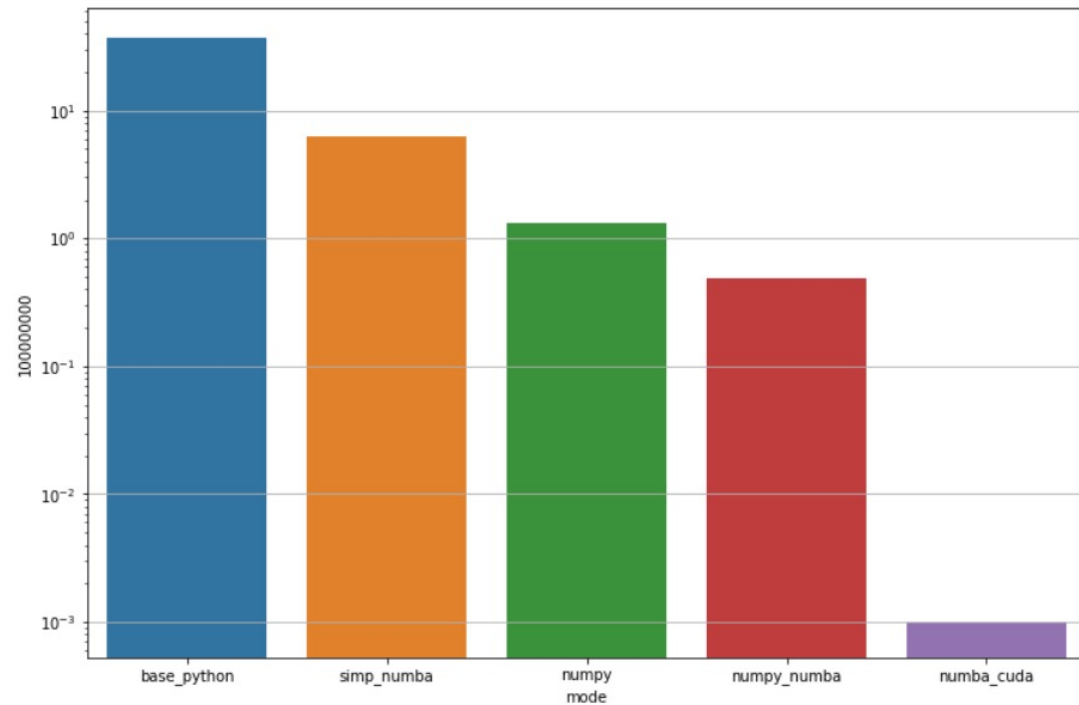
@vectorize([float64(float64)], target='cuda')
def logistic_numba_v_cuda(y):
    return 1 / (1 + math.exp(y))
```

Бэнчмарк применения логистического преобразования

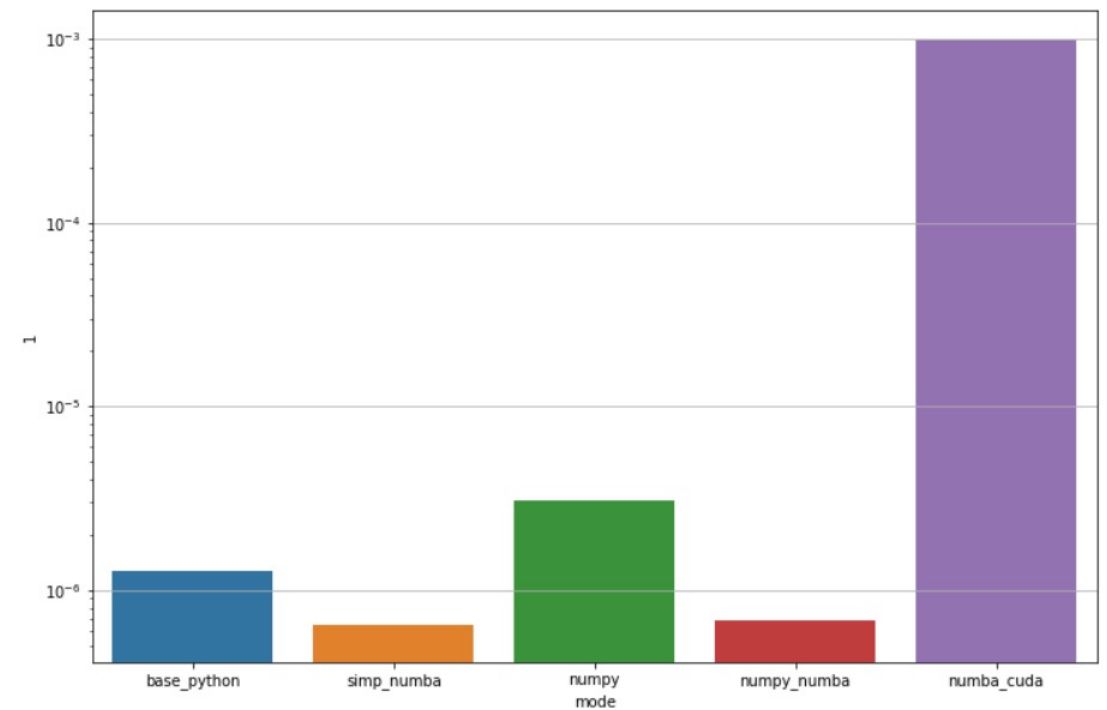


Бэнчмарк применения логистического преобразования

10^8 чисел



1 число



Декораторы Numba

@guvectorize

В то время как `vectorize()` позволяет писать ufuncs, которые работают с одним элементом за один раз, декоратор `guvectorize()` делает еще один шаг вперед и позволяет писать ufuncs, которые будут работать с произвольным числом элементов входных массивов, а также принимать и возвращать массивы различных размеров. Например, он полезен при расчете скользящего среднего.

```
@guvectorize(['void(float64[:], int32, float64[:])'],
             '(n),()->(n)', target='cuda')
def moving_average_cuda(vector, window, out):
    for i in range(len(vector)):
        acc = 0.
        values = vector[i:i+window]
        for val in values:
            acc += val
        out[i] = acc / len(values)
```

Слишком тяжело

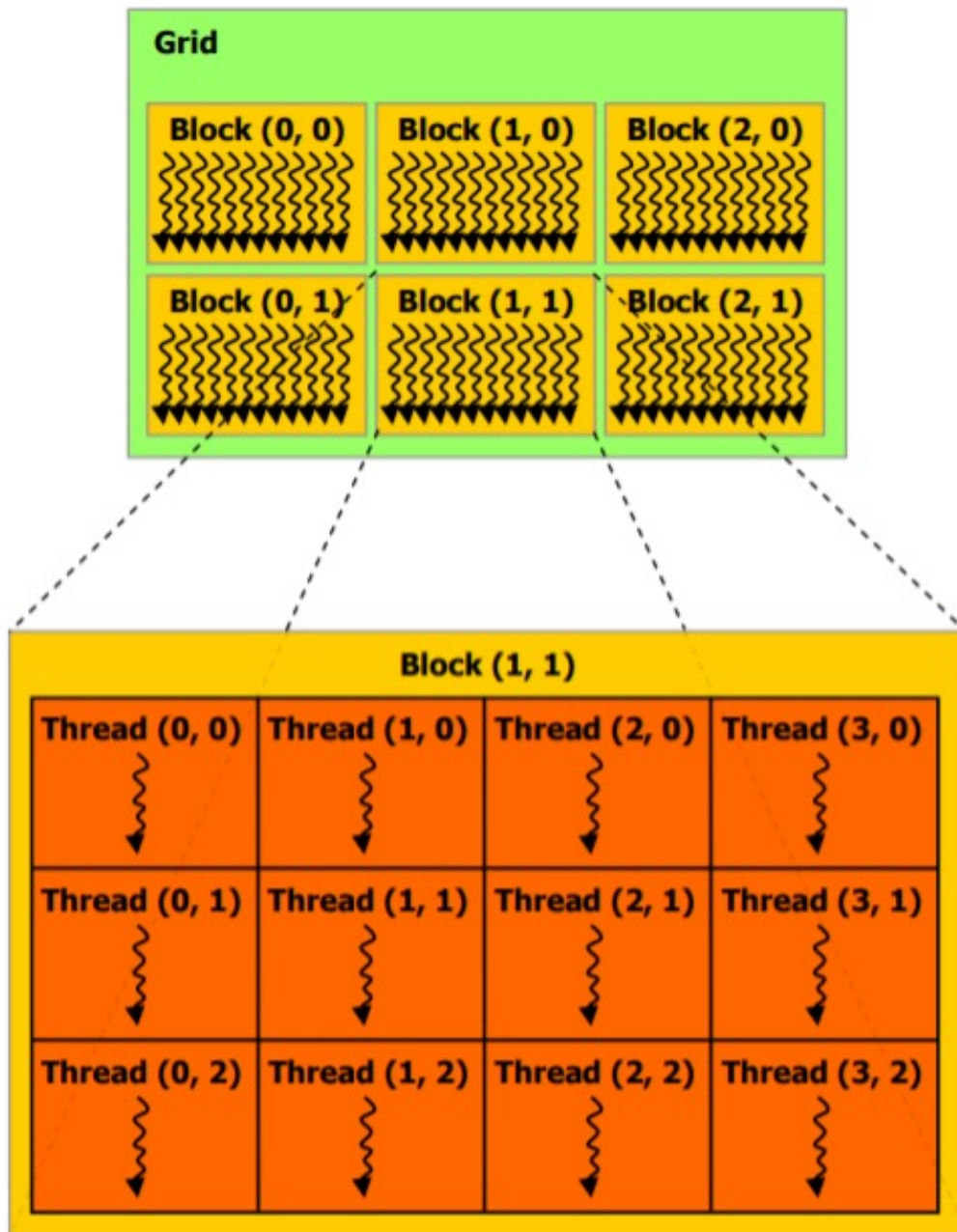


До свидания

Декораторы Numba

@cuda.jit

- Вычисления на GPU отличаются от привычного процесса на CPU. Код выполняется большим количеством потоков одновременно с учетом иерархии сетки вычислений, блоков и самих потоков.
- Основа – функция ядра (kernel function).
 1. Функция ядра не может в явном виде возвращать значение, она может только записать его в переданный функции массив;
 2. Функции ядра явно определяют иерархию потоков при вызове: количество блоков и количество потоков на блок.



@cuda.jit

Основные шаги:

- Создание экземпляра kernel function, указав количество блоков на сетку и количество потоков на блок. Их произведение даст общее количество запущенных потоков.
- Запуск ядра, передавая ему входной массив (и любые отдельные выходные массивы, если это необходимо). По умолчанию запуск ядра выполняется синхронно: функция возвращается, когда ядро завершает выполнение и данные синхронизируются обратно.

Grid(сетка) – совокупность всех потоков, запущенных в рамках одной задачи, выполняющих одинаковую операцию «независимо».

@cuda.jit

1-d array:

```
threadsperblock = 256  
blockspergrid = math.ceil(data.shape[0] / threadsperblock)  
my_kernel[blockspergrid, threadsperblock>(*args)
```

2-d array:

```
threadsperblock = (16, 16)  
blockspergrid_x = int(math.ceil(x.shape[0] / threadsperblock[0]))  
blockspergrid_y = int(math.ceil(y.shape[1] / threadsperblock[1]))  
blockspergrid = (blockspergrid_x, blockspergrid_y)  
my_kernel[blockspergrid, threadsperblock>(*args)
```

Помимо переменных функции также необходимо передать массив, в который будет записан ответ.

Выводы

Плюсы:

- 1) Хорошо оптимизирует CPU код, отлично оптимизирует циклы
- 2) Легко применять
- 3) Позволяет производить вычисления на GPU
- 4) Поддерживает и хорошо оптимизирует многие функции библиотеки Numpy, в том числе легко распараллеливает код, если это возможно
- 5) Вроде как есть @roc.jit для AMD видеокарт
- 6) Больше данных – больше скорость

Минусы:

- 1) Не любит наш любимый list, ругается, если в нем объекты разных типов
- 2) Поддерживает только часть объектов и функций Python, что-то отдельно реализовано в numba
- 3) Написание функций на GPU сильно отличается от CPU
- 4) При первом вызове функции происходит компиляция, что замедляет код => «подогреваем кэш» и вводим строгую типизацию
- 5) Делать return внутри цикла не стоит

P.S. Разогнанная функция может быть вызвана внутри обычной функции, но внутри разогнанной могут быть только также разогнанные функции.



О чем не поговорили?

- Поддержка классов в Numba `@jitclass`
- `@generated_jit` если нужна функция, поведение которой зависит от типа входной переменной
- `@cfunc`, `@overload`, `@stencil`
- AMD видеокарты
- AOT компиляция